

[Bài đọc] Các mã lỗi trong RESTful API

1. Mở đầu

- Trong phát triển dịch vụ web, việc giao tiếp giữa **client** và **server** luôn cần một cơ chế rõ ràng để xác định trạng thái xử lý của mỗi yêu cầu. **Mã lỗi HTTP (HTTP status code)** chính là ngôn ngữ chung giúp hai bên hiểu nhau: yêu cầu có thành công hay không, nếu thất bại thì nguyên nhân là gì
- Việc nắm vững các mã lỗi và áp dụng đúng chuẩn không chỉ giúp **client** xử lý tình huống chính xác, mà còn giúp **lập trình viên** dễ dàng debug, bảo trì và nâng cao trải nghiệm người dùng

2. Phân loại mã lỗi

- Mã lỗi trong RESTful API được chia thành 5 nhóm chính:
 - **1xx – Thông tin (Informational)**: Cho biết server đã nhận yêu cầu và đang tiếp tục xử lý
 - **2xx – Thành công (Success)**: Yêu cầu đã được xử lý thành công (200 OK, 201 Created)
 - **3xx – Chuyển hướng (Redirection)**: Client cần thực hiện yêu cầu ở một địa chỉ khác (301 Moved Permanently, 302 Found)
 - **4xx – Lỗi phía Client (Client Error)**: Yêu cầu có vấn đề từ phía client, như dữ liệu sai định dạng hoặc không có quyền truy cập (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found)
 - **5xx – Lỗi phía Server (Server Error)**: Server gặp sự cố khi xử lý yêu cầu (500 Internal Server Error, 503 Service Unavailable)

3. Các mã lỗi phổ biến trong RESTful API

- **400 Bad Request**: Dữ liệu gửi lên sai định dạng, thiếu tham số hoặc không hợp lệ
- **401 Unauthorized**: Người dùng chưa xác thực, cần đăng nhập hoặc gửi token hợp lệ

- **403 Forbidden:** Người dùng đã xác thực nhưng không có quyền thực hiện hành động
- **404 Not Found:** Tài nguyên được yêu cầu không tồn tại
- **429 Too Many Requests:** Client gửi quá nhiều yêu cầu trong một khoảng thời gian (rate limiting)
- **500 Internal Server Error:** Lỗi xử lý phía server, thường do bug hoặc exception chưa được xử lý

4. Quy trình xử lý lỗi trong API

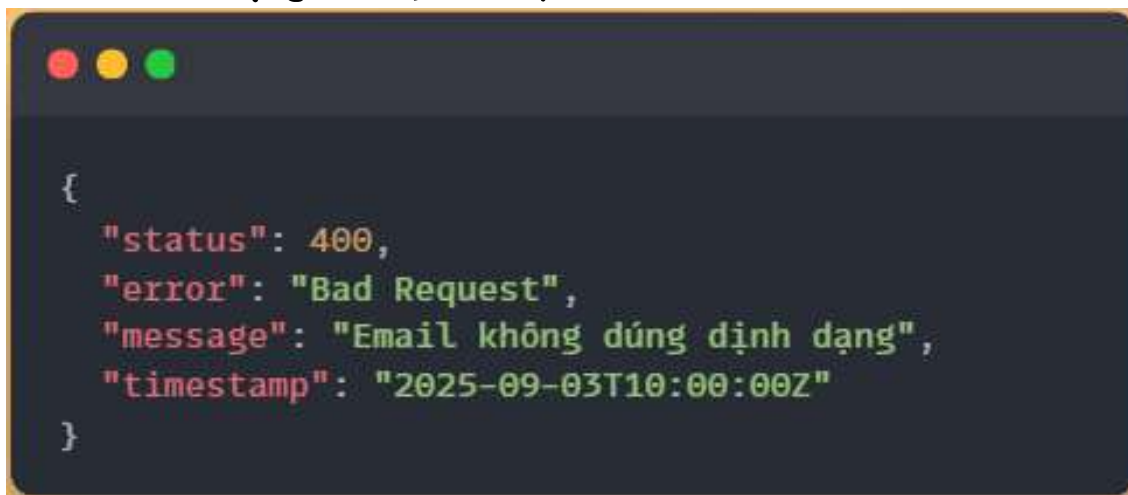
- Một quy trình chuẩn khi xử lý lỗi trong RESTful API gồm 3 bước:
 - **Xác định lỗi:** Nhận mã lỗi từ server và xác định nguyên nhân
 - **Thông báo lỗi:** Trả thông tin rõ ràng, dễ hiểu cho client hoặc người dùng
 - **Khôi phục yêu cầu:** Gợi ý cách sửa lỗi (ví dụ: đăng nhập lại với lỗi 401, kiểm tra dữ liệu nhập với lỗi 400)

5. Bảng biểu so sánh

Nhóm mã lỗi	Ý nghĩa chính	Ví dụ phổ biến	Tình huống thực tế
1xx (Thông tin)	Server đang xử lý yêu cầu	100 Continue	Upload file lớn, server báo tiếp tục gửi dữ liệu
2xx (Thành công)	Yêu cầu thành công	200 OK, 201 Created	Lấy danh sách sản phẩm, tạo user mới
3xx (Chuyển hướng)	Cần gửi yêu cầu đến URL khác	301 Moved Permanently	Website đổi domain, chuyển hướng đến link mới
4xx (Lỗi client)	Sai sót từ phía client	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests	Nhập sai dữ liệu, chưa đăng nhập, không có quyền, truy cập tài nguyên không tồn tại, spam request
5xx (Lỗi server)	Server gặp sự cố	500 Internal Server Error, 503 Service Unavailable	Lỗi logic, crash hệ thống, server quá tải bảo trì

6. Kiến thức mở rộng

- Tùy biến thông báo lỗi: Ngoài mã lỗi, API nên trả về thông báo chi tiết dưới dạng JSON, ví dụ:

A screenshot of a code editor with a dark background and orange border. It displays a JSON object representing an error response. The JSON has four key-value pairs: 'status' with value 400, 'error' with value 'Bad Request', 'message' with value 'Email không đúng định dạng', and 'timestamp' with value '2025-09-03T10:00:00Z'. The code is color-coded: strings are green, numbers are orange, and punctuation is white.

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Email không đúng định dạng",
  "timestamp": "2025-09-03T10:00:00Z"
}
```

- **Best Practices** khi xử lý lỗi trong REST API:
 - Sử dụng mã lỗi **chuẩn HTTP** thay vì tự đặt
 - Luôn trả về **thông báo rõ ràng** cho client
 - Áp dụng **rate limiting** với mã 429 để chống spam
 - Với lỗi 500, tránh trả chi tiết kỹ thuật nhạy cảm (stack trace), chỉ nên log trong server
- **So sánh REST với GraphQL về xử lý lỗi:**
 - REST dựa nhiều vào mã HTTP
 - GraphQL thường trả về 200 OK, nhưng lỗi được chứa trong field **errors** của response JSON